

INFORMATICA - 3 LICEO SCIENTIFICO - MODULO 1 - TEORIA

Programmazione in Python3

Revisione: 04/10/2026

Python

Python ha due macro versioni (2.7 e 3), noi usiamo sempre la versione 3. Quindi quando parliamo di python in realta' intendiamo python3.

Python e' un nome che si riferisce a due cose distinte:

- un linguaggio di programmazione
- un interprete

Linguaggio di programmazione

Un linguaggio di programmazione come per la lingua italiana prevede una **grammatica**, un insieme di regole per costruire delle frasi (o delle istruzioni nel caso di python) che compongono un testo (o un programma nel caso di python).

Python prevede una **sintassi** rigorosa che deve rispettare le regole grammaticali definite dal linguaggio di programmazione.

I linguaggi nascono per permettere a due entita' di comunicare, nel parlato le entita' sono due persone mentre nei linguaggi di programmazione le entita' sono una persona e un calcolatore (un computer).

Un linguaggio di programmazione usa un **file di testo** (**.py**) al posto della voce nel parlato. Le istruzioni devono essere scritte in un file di testo (codice) e devono essere **interpretate** (parsing) ed **eseguite** sul calcolatore dall' **interprete**.

Interprete

Python si dice linguaggio interpretato in quanto per poterlo utilizzare bisogna installare l'interprete di python sul proprio calcolatore.

L'interprete si puo' scaricare al seguente link <https://www.python.org/downloads/>

L'interprete di python e' un **programma eseguibile** installato su un calcolatore al pari degli altri programmi/app che vengono installati in un computer (Calcolatrice, Chrome, Outlook, ...).

Quando il programmatore ha finito di scrivere il codice (nel file `.py`) lo può eseguire invocando l'interprete di python e passandogli in input il file appena scritto.

Per prima cosa l'interprete di python controlla che nel file non ci siano errori sintattici, se ci sono degli errori sintattici si blocca e restituisce un'eccezione (`compile time error`).

Se il file non contiene errori sintattici l'interprete esegue riga per riga le istruzioni del codice sorgente; in questa fase l'interprete legge una riga, la converte in codice macchina (binario) e la esegue sul calcolatore dando vita ad un nuovo processo.

Compilatore

Altri linguaggi di programmazione noti come C, C++, C#, Java, etc. sono linguaggi compilati, non interpretati.

La differenza principale è che in un linguaggio compilato tutto il codice sorgente viene letto (parsing), convertito in codice macchina e salvato in un file eseguibile (`.exe`, `.out`, ...). Il file eseguibile è tipicamente offuscato (non si può più ricostruire il codice sorgente) e può essere rilasciato, per esempio da un'azienda o uno sviluppatore, ai clienti/utenti. Dopo il rilascio (in un secondo momento) l'utente finale può eseguire il programma sul suo calcolatore. I linguaggi compilati sono tipicamente molto più veloci (talvolta anche di un fattore x100) ed efficienti dei linguaggi interpretati ma sono più rigorosi e lenti da sviluppare.

Nel linguaggio interpretato invece si usa sempre il codice sorgente che non viene mai compilato in un programma eseguibile. Il codice sorgente viene letto una riga alla volta ed eseguito istantaneamente sempre una riga alla volta. Questo permette per esempio di modificare il codice sorgente durante l'esecuzione (solo in alcuni linguaggi). I linguaggi interpretati sono più lenti ma molto più user-friendly e apprezzati per lo scripting, la ricerca, lo sviluppo di prototipi (fast-prototyping), applicazioni AI. Python fra tutti è fra i più usati al mondo per gli ambiti sopra elencati. Un altro linguaggio interpretato molto famoso è JavaScript che dà vita a tutte le pagine web dinamiche che esistono.

IDE

Un IDE è un ambiente di sviluppo ovvero un'applicazione locale o in cloud che lo sviluppatore utilizza per scrivere il codice e testarlo prima del rilascio in produzione. Uno degli IDE più famosi ed utilizzati in tutto il mondo è Visual Studio Code (VS Code). Questo IDE gratuito e open source prodotto da Microsoft è particolarmente utile ed apprezzato per la sua versatilità: permette di sviluppare in qualsiasi linguaggio esistente (Python, C, C++, C#, Java, JavaScript, SQL, HTML, ...) e si integra con qualsiasi strumento di sviluppo esistente.

Primo programma

Il primo programma di uno sviluppatore python è il seguente

```
print("hello world!")
```

Per prima cosa il codice deve essere salvato in un file `.py` per esempio `hello_world.py`.

Successivamente bisogna scegliere un ambiente di sviluppo che puo' essere locale (sul proprio computer) o in cloud (su servizi cloud); le principali alternative sono:

- Sviluppo in locale (sul proprio computer) installando Python3 e Visual Studio Code sul proprio computer (Windows, Mac o Linux)
- Sviluppo in cloud tramite account Google lavorando su Google Drive con Colaboratory (funziona anche da tablet e smartphone)
- Sviluppo in cloud tramite account GitHub lavorando su GitHub CodeSpaces (funziona anche da tablet e smartphone)
- Sviluppo in cloud tramite account Microsoft lavorando su Microsoft OneDrive (funziona anche da tablet e smartphone)

Una volta scelto l'ambiente di sviluppo per eseguire il programma bisogna invocare l'interprete sul file `hello_world.py`:

- in Visual Studio Code premere sul triangolo in alto a destra oppure lanciare l'istruzione da terminale `python3 hello_world.py`
- in Google Colab premere sul triangolo verde della cella di codice
- ...

Se il programma e' stato eseguito correttamente apparira' una finestra terminale con la scritta `"hello world!"`.

Complimenti, ora sei ufficialmente un programmatore Python!

Commenti

In python i commenti al codice si fanno con `#`; tutto cio' che viene scritto a destra di un `#` non viene interpretato da python ed e' come se non esistesse/non fosse mai stato scritto, e' un commento.

Variabili

In python si possono dichiarare delle variabili per salvare nella memoria volatile del calcolatore (memoria RAM) dei valori da riutilizzare nel programma.

Le variabili sono il primo concetto da apprendere in un linguaggio di programmazione perche' ci permettono di gestire la memoria allocata dal nostro programma. Senza memoria un programma non puo' fare quasi nulla allo stesso modo del cervello umano.

Dichiarazione di una variabile

```
x: int = 123
```

La dichiarazione di una variabile e' composta da tre sezioni:

- nome della variabile (`x`)
- tipo della variabile (`int`)
- valore della variabile (`123`)

Il nome della variabile e' un nome mnemonico a scelta:

- puo' contenere solo caratteri alfa-numeric e underscore (`_`)
- deve iniziare con una lettera o un underscore (`_`)
- deve essere in `lower_snake_case` per convenzione

Il nome e' separato dal tipo tramite `:` (un separatore), il tipo di una variabile ne decreta la natura. I tipi fondamentali sono:

- `int` (numero intero senza virgola)
- `float` (floating point, numero con la virgola)
- `bool` (booleano, afferente alla logica, puo' assumere solo due valori `True` e `False`)
- `str` (string, una sequenza di caratteri come un caratter, una parola, una frase, ...)

Il tipo di una variabile serve in prima battuta a decretare la codifica del valore della variabile in binario. Considerato che il valore deve essere salvato in memoria RAM, deve essere applicata una codifica che converta il valore "human-readable" in valore binario "machine-readable".

Per esempio consideriamo il valore `int 8` sapendo che esso e' un intero senza segno una codifica possibile sarebbe la conversione del numero 8 in base 10 al numero 100 in base 2 (binario).

Se invece consideriamo il valore `bool True` questo potrebbe essere convertito in intero `1` (mentre `False` verrebbe convertito in intero `0`) e poi a sua volta convertito in binario `1`.

Infine se consideriamo il valore `str a` questo potrebbe essere convertito in binario tramite una codifica/tabella di conversione come ASCII, UTF-8, etc. Se per esempio utilizzassimo la codifica ASCII su 1 byte il carattere `a` diventa `01100001`.

La dichiarazione di una variabile prevede solamente nome e tipo della variabile; queste informazioni sono sufficienti al linguaggio per `allocare` in memoria RAM uno spazio di memoria capace di contenere che assegneremo in seguito alla variabile.

Assegnazione di una variabile

Data una variabile dichiarata e' possibile assegnare un valore con l'operatore `=`

```
x = 1
x = 2
```

Per esempio sopra viene assegnato alla variabile `x` un primo valore `1` quindi in memoria RAM, nella cella di memoria puntata dalla variabile `x`, verra' scritto il valore `1` codificato in binario (per esempio su un byte avremmo: `00000001`). Nella riga successiva la variabile `x` viene ri-assegnata

ad un secondo valore `2` : una riassegnazione comporta la sovrascrittura del valore `2` (in binario `00000010`) nella cella di memoria puntata dalla variabile `x`. In questo caso il valore precedente `1` e' perso per sempre anche se nel codice sorgente e' dichiarato nella prima riga il programma non puo' piu' ricordarsi di quel valore perche' e' stato sovrascritto.

E' possibile, nella stessa istruzione, dichiarare una variabile ed assegnarle un valore

```
x: int = 123
```

E' molto importante ricordarsi le tre componenti fondamentali di una variabile:

- nome
- tipo
- valore

Esse sono separate da due separatori (`:` e `=`) e c'e' una spaziatura convenzionale da seguire.

Evitare di usare le parentesi tonde e spazi extra se non sono necessari, il codice deve seguire non solo le regole sintattiche di python ma anche delle buone pratiche per essere leggibile da tutti e facilmente comprensibile

Il nome delle variabili deve essere significativo per chiunque legga il codice, deve essere in `lower_snake_case` e dovrebbe essere scritto in `inglese` (all'inizio va bene scrivere codice in italiano per facilitare la comprensione ma ricordatevi sempre che un buon codice deve essere scritto interamente in inglese).

Stampa

Per stampare un valore costante o il contenuto di una variabile si usa la funzione `print()`

```
print(1)
print(True)
print(3.14)
print("hello world!")
```

```
x: int = 1
print(x)
```

Esempi

```
# dichiara una variabile "x" di tipo intero ("int")
# con valore 314
x: int = 314

# dichiara una variabile "y" di tipo numerico
# con virgola ("float") con valore 3.14
y: float = 3.14

# dichiara una variabile "z" di tipo booleano ("bool")
# con valore True
z: bool = True

# dichiara una variabile "w" di tipo testo/stringa ("str")
# con valore "Hello World!"
w: str = "Hello World!"

# stampa il valore di x, y, z e w
print(x)
print(y)
print(z, w)
```

```
# dichiara una variabile "a" di tipo intero
a: int

# assegna ad a il valore 1
a = 1

# stampa il valore di a
print(a)

# riassegna ad a il valore 2
a = 2

# stampa il valore di a
print(a)

# riassegna ad a il valore 3
a = 3

# stampa il valore di a
print(a)

# stampa il valore iniziale di a

# non si puo' stampare, il valore iniziale di a si e' perso!
```

Aritmetica degli interi

E' possibile, come in matematica, usare l'aritmetica degli interi; e' possibile usare molti operatori matematici (+, -, *, /, ...) con le variabili intere `int`.

Esempi

```
# dichiara una variabile "x" di tipo intero ("int") con valore 1
x: int = 1

# dichiara una variabile "y" di tipo intero ("int") con valore 2
y: int = 2

# dichiara una variabile "z" come somma di x e y
z: int = x + y

# stampa il valore di "x", "y" e "z"
print(x, y, z)

# aggiorna il valore di "x" a 2
x = 2

# stampa il valore di "x", "y" e "z"
print(x, y, z)
```

```
x: int = 1 + 1    # somma
print(x)

x: int = 2 - 2    # sottrazione
print(x)

x: int = 3 * 3    # moltiplicazione
print(x)

x: int = 4 // 4   # divisione intera (arrotondamento)
print(x)

x: int = 5 % 4    # resto della divisione intera
print(x)

x: int = 3 ** 3   # elevamento a potenza "3 alla terza"
print(x)
```

```
# predici il valore di w
w = (((2 + 1) // 3 + (2 - 2) * 1) ** 2) % 10
print(w)
```

Aritmetica dei float

Identica a quella degli `int` salvo per piccole caratteristiche ovvie come l'uso della divisione "normale" `/` anziche' la divisione intera `//`

Aritmetica booleana

Le variabili booleane seguono la logica di Bool, possono assumere solo due valori logici: `False` (0) e `True` (1).

I principali operatori logici che si possono usare con le variabili booleane sono: `and` (e), `or` (o) e `not` (negazione).

Tabelle di verita'

Un'espressione booleana ha una tabella di verita', per esempio la tabella dell' `and`

```
# True and True = True
# True and False = False
# False and True = False
# False and False = False
```

la tabella dell' `or`

```
# True or True = True
# True or False = True
# False or True = True
# False or False = False
```

la tabella nel `not`

```
# not True = False
# not False = True
```

A partire dalle tabelle soprastanti e' possibile stilare la tabella di verita' di qualsiasi espressione come per esempio `not(a and b) or c`

```
not(True and True) or True = True
not(True and True) or False = False
not(True and False) or True = True
not(True and False) or False = True
not(False and True) or True = True
not(False and True) or False = True
```

```
not(False and False) or True = True
not(False and False) or False = True
```

Essendo i booleani valori di verita' essi possono essere creati tramite confronti fra altri valori (anche non necessariamente booleani).

```
b: bool = 10 > 7 # True
c: bool = 10 <= 7 # False
d: bool = 10 == 10 # True
```

Da notare che il confronto uguale prevede l'uso di doppio carattere uguale `==` per differenziare il confronto dall'assegnazione (uguale singolo `=`) di un valore ad una variabile

Esempi

```
# dichiara una variabile "x" di tipo booleano ("bool")
# con valore True
x: bool = True

# dichiara una variabile "y" di tipo booleano ("bool")
# con valore False
y: bool = False

# stampa il valore di "x" e "y"
print(x, y)
```

```
# dichiara una variable "z" di tipo booleano come x OR y
# e stampane il valore
z: bool = x or y
print(z)

# assegna alla variable "z" il valore x AND y e stampala
z = x and y
print(z)

# assegna alla variable "z" il valore 1 > 2 e stampala
z = 1 > 2
print(z)

# assegna alla variable "z" il valore 2 > 1 e stampala
z = 2 > 1
print(z)

# assegna alla variable "z" il valore 1 <= 2 e stampala
```

```
z = 1 <= 2
print(z)

# assegna alla variable "z" il valore 2 <= 1 e stampala
z = 2 <= 1
print(z)

# assegna alla variable "z" il valore
# (2 <= 1) AND (x) OR (x AND y) e stampane il valore
z = (2 <= 1) and (x) or (x and y)
print(z)

# assegna alla variabile "Z" il confronto fra due stringhe
z = "ciao" == "hello"
print(z)
```

Operazioni con le stringhe

E' possibile sommare due stringhe? o dividere due stringhe? chiaramente no, almeno non nel senso matematico del termine.

Su una stringa possiamo comunque applicare delle operazioni per esempio la concatenazione e la divisione, l'indicizzazione dei caratteri, ...

Immaginiamo sempre una stringa come una sequenza ordinata di caratteri, i quali sono indicizzati in modo incrementale a partire da 0

```
stringa: hello bob!
indice : 0123456789
```

In questo caso il carattere di indice **6** sara' la **b** di **bob**.

Esempi

```
# dichiara una variabile "x" di tipo stringa ("str")
# con valore "Hello"
x: str = "Hello"

# dichiara una variabile "y" di tipo stringa ("str")
# con valore "World!"
y: str = "World!"

# stampa il valore di "x" e "y"
print(x, y)

# dichiara due stringhe in-linea utilizzando
```

```
# le 2 dichiarazioni ammesse: "...", '...'  
a: str = "hello world!"  
b: str = 'hello world!'  
  
# dichiara due stringhe multi-linea utilizzando  
# le 2 dichiarazioni ammesse: """...""", '''...'''  
c: str = """this is a  
multi-line  
string"""  
  
d: str = '''this is another  
multi-line string'''  
  
# dichiara una variabile "z" di tipo stringa come  
# concatenazione di x e y (con uno spazio in mezzo)  
z: str = x + y  
  
# ridichiara "z" usando "string format" (f"  
# per concatenare x e y  
z = f"{x}{y}"  
  
# stampa il primo carattere di z  
print(z[0])  
  
# stampa il secondo carattere di z  
print(z[1])  
  
# stampa i primi due caratteri di z  
print(z[0:2])  
print(z[:2])  
  
# stampa l'ultimo carattere di z  
print(z[-1])  
  
# stampa gli ultimi tre caratteri di z  
print(z[-3:-1])  
print(z[-3: ])  
  
# concatena i primi tre caratteri di z agli ultimi due,  
# salva il valore in w e stampa la variabile w  
w: str = z[:4] + z[-2:]  
print(w)  
  
# dichiara una variabile booleana b come confronto fra w e "Held!"  
b: bool = w == "Held!"  
  
# assegna alla variabile booleana b il confronto fra w e "He!"  
b = "He!" == w  
  
# assegna alla variabile booleana b il confronto "w contiene  
# la sottostringa "World" usando l'operatore in  
b = "World" in w
```

Casting

Il ``cast`` e' un operazione che permette di trasformare un valore/variabile di un certo tipo in un altro tipo diverso. Per esempio potrei voler convertire:

- * il valore ``int`` ``1`` in un ``bool`` (``True``)
- * il valore ``int`` ``1`` in una ``str`` (``"1"``)
- * il valore ``str`` ``"1"`` in un ``int`` (``1``)
- * il valore ``float`` ``3.0`` in un ``int`` (``3``)
- * ...

Il cast prevede di usare un tipo dato come funzione di conversione

```
```py
i: int = 1
b: bool = bool(i)

b = bool(1)
```

## funzione `type`

E' possibile ottenere il tipo di un valore/variabile tramite l'uso della funzione `type()`

```
print(type(3.14)) # float
```

## Esempi

```
dichiara una variabile "x" di tipo int valore 1
x: int = 1

usa il cast per trasformare il valore di x in float, bool e str
f: float = float(x)
b: bool = bool(x)
s: str = str(x)

stampa il tipo di x, f, b, s usando la funzione type
print(type(x))
print(type(f))
print(type(b))
print(type(s))

ri-converti f, b, s in int
int(f)
int(b)
int(s)
```

```
il cast e' sempre ammesso? posso convertire "abc" in int?
no, l'istruzione int("abc") genera un errore nell'interprete
```

## Funzioni

---

Le funzioni sono uno strumento in python per delimitare un blocco di codice, parametrizzarlo e poterlo riutilizzare in vari contesti.

I concetti chiave sono che la funzione delimita un blocco di codice che seguirà una indentazione (tabulazione) diversa dal resto di codice; e che la funzione è riutilizzabile e configurabile tramite dei parametri.

```
def somma(a: int, b: int) -> int:
 ...
```

La firma di una funzione è la prima riga, essa deve iniziare con la keyword `def` seguita dal nome della funzione. La firma di una funzione dichiara anche variabili in input e valore di ritorno (output).

```
def somma(a: int, b: int) -> int:
 c: int = a + b
 return c

risultato: int = somma(1, 2)
print(risultato)
```

Il corpo della funzione comprende tutte le righe che seguono la firma e che vengono indentate a destra di almeno un `tab`. La keyword `return` precede il valore di ritorno della funzione (che deve essere coerente con il tipo dichiarato nella firma della funzione). Il `return` determina la fine della funzione e tipicamente è nell'ultima riga. In alcuni casi ci possono essere più di un `return` nel caso di più blocchi di codice annidati.

```
def check(eta: int) -> str:
 if eta >= 18:
 return "maggiorenne"
 elif 0 <= eta < 18:
 return "minorenne"
 return "eta' invalida"
```

Nell'esempio notiamo che ci sono varie diverse indentazioni e quindi vari blocchi di codice (3 blocchi in totale).

Una funzione può non avere tipo di ritorno e di conseguenza `return` in questo caso si parla di procedura.

```
def log(messaggio: str):
 print(messaggio)
```

## Esempi

```
dichiara una funzione somma
che prenda in input due argomenti a e b di tipo int
e restituisca in output un valore di tipo int
def somma(x: int, y: int) -> int:
 z: int = x + y
 return z

dichiara le funzioni: sottrai, dividi e moltiplica
def sottrai(x: int, y: int) -> int:
 z: int = x - y
 return z

def dividi(x: int, y: int) -> float:
 z: float = x / y
 return z

def moltiplica(x: int, y: int) -> int:
 z: int = x * y
 return z

dichiara una funzione "concatena" che date due stringhe in input
le concatena e ritorna in output il valore concatenato

def concatena(x: str, y: str) -> str:
 z: str = x + y
 return z

dichiara una funzione "divisore" che dati in input
due numeri interi x e y, controlla se x è un divisore intero di
y
e ritorna il risultato come valore di tipo booleano

def divisore(x: int, y: int) -> bool:
 return y % x == 0

qual'è la firma delle funzioni "concatena" e "divisore"?

def concatena(x: str, y: str) -> str:
def divisore(x: int, y: int) -> bool:

cos'è print? qual'è la firma di print?
```

```
print e' una funzione che prende in input una variabile generica
stampa in console il valore della variabile
non ha tipo di ritorno

def print(arg: any):
 ...
```

## Scope

---

Consideriamo questo esempio:

```
print("inizio programma")
x: int = 1
y: int = 2

def somma(x: int, y: int) -> int:
 print("funzione somma in esecuzione...")
 w: int = x + y
 return w

z = somma(3, 4)

cosa verra' stampato nelle prossime righe?
print(x)
print(y)
print(z)
```

Nell'esempio notiamo che ci sono due dichiarazioni delle variabili `x` e `y`: una dichiarazione iniziale con valori `x=1` e `y=2` ed una seconda dichiarazione nella funzione `somma` come `input x: int, y: int`. A seguire, dopo il corpo della funzione, viene invocata la `somma` con passaggio valori `3,4`.

Il passaggio valori prevede che alla variabile `x` della funzione `somma` venga assegnato il primo valore `3` mentre alla variabile `y` il secondo valore `4`.

Infine vengono stampati i valori di `x` e di `y`; il dubbio che sorge e' se verra' stampato il valore iniziale (`x=1, y=2`) o il valore del passaggio parametri della funzione (`x=3, y=4`).

Per capire come funziona questo esempio dobbiamo introdurre il concetto di `scope` di una variabile; una variabile "vive" all'interno di un blocco di codice e viene deallocata (rimossa dalla memoria RAM) quando il programma esce dal blocco di codice di dichiarazione.

Se, come in questo caso, una funzione ri-dichiara una variabile con lo stesso nome (omonima) di una variabile pre-esistente all'esterno della funzione, pur avendo lo stesso nome le due variabili possono co-esistere (in memoria verranno allocati due spazi ad-hoc per ognuna di esse), non

andranno in conflitto, non si sovrascriveranno, a tutti gli effetti e' come se avessero due nomi diversi.

## Pacchetto

---

Quando si iniziano a scrivere grandi quantita' di codice e programmi complessi emerge il tema della mantenibilita' del codice: un codice scritto in un unico file da migliaia di righe non e' mantenibile e rende difficile la collaborazione in un team di sviluppo dove varie persone diverse scrivono il codice una singola applicazione.

Per questi motivi e anche per poter condividere il proprio codice e lavoro con altre persone sono stati creati i moduli (file), pacchetti (cartelle) e le librerie (codice condiviso).

Un package in python non e' altro che una cartella, il nome della cartella corrisponde al nome del pacchetto. Un package puo' contenere vari file `.py`. Un package deve contenere un file vuoto chiamato `__init__.py` (questo e' un tema avanzato che verra' ripreso piu' avanti).

## Import

Quando un package o un modulo viene condiviso su piu' progetti o comunque non e' salvato dentro la stessa cartella del nostro codice sorgente viene chiamato in gergo libreria.

Una libreria deve essere importata per poter essere utilizzata, per esempio poniamo caso che io voglia usare la funzione `getsizeof` dichiarata nel modulo `sys`.

```
import sys
print(sys.getsizeof("hello world!"))
```

La funzione `getsizeof` ritorna il numero di bytes occupati in memoria da un valore/variabile.

Python permette di importare l'intera libreria e di usare il suo nome per accedere a tutte le sue funzioni; oppure, nel caso in cui abbia bisogno di una singola funzione, permette di importare una funzione specifica.

```
from sys import getsizeof
print(getsizeof("hello world!"))
```

## Esempi

```
dal modulo sys importa la funzione getsizeof
from sys import getsizeof

stampa il risultato di getsizeof(1)
```

```
size: int = getsizeof(1)
print(size)
```

## Input

---

In python esiste una funzione built-in chiamata `input()` essa permette di bloccare il flusso di esecuzione del programma e chiedere all'utente di inserire un valore nel terminale.

L'utente inserisce il valore e preme il tasto invio/enter, a quel punto la funzione `input()` ritorna in output il valore (stringa) inserito dall'utente e il programma continua l'esecuzione.

```
chiedi all'utente in input la sua eta e salvala nella variabile
intera x usando la funzione built-in "input"

eta_stringa: str = input("Inserire la propria eta': ")
eta: int = int(eta_stringa)
print(eta)

fai lo stesso usando il cast direttamente sull'input
eta: int = int(input("Inserire la propria eta': "))
print(eta)
```

## If

---

In qualsiasi linguaggio di programmazione esistono due costrutti fondamentali per implementare un algoritmo: la selezione e l'iterazione. Qualsiasi linguaggio che possiede questi due costrutti si dice linguaggio Turing-completo e puo' implementare qualsiasi algoritmo esistente.

Il primo concetto fondamentale da imparare e' la selezione (`if`) che permette di dividere il codice in due o piu' flussi esclusivi. Grazie a questa funzionalita' possiamo spartire il flusso di esecuzione in due o piu' strade da percorrere e solo una di queste verra' eseguita (quella che rispetta una data condizione), le altre verranno saltate.

```
print("inizio del programma")
eta: int = 22
if eta < 18:
 print("sei minorenne")
else:
 print("sei maggiorenne")

print("termine del programma")
```

Nell'esempio sopra riportato non sara' mai possibile stampare sia "sei minorenni" che "sei maggiorenni" in quanto l' `if` divide il flusso di esecuzione in base alla condizione `eta < 18`.

## Esempi

```
se la variabile x >= 18 stampa "sei maggiorenne"
se la variabile x < 18 stampa "sei minorenni"
se la variabile x > 50 stampa "sei over cinquanta"

eta: int = 54
if eta < 18:
 print("sei minorenni")
elif eta < 50:
 print("sei maggiorenne")
else:
 print("sei over-50")
```

## while

---

Il secondo costrutto fondamentale dell'algorithmica e' l'iterazione o ciclo; un ciclo permette di eseguire piu' volte, iterativamente, un blocco di codice.

Poniamo caso che io voglia stampare i numeri da 1 a 5.

```
print(1)
print(2)
print(3)
print(4)
print(5)
```

Se ora volessi stampare i numeri da 1 a 1000 mi servirebbero ben 1000 righe di codice e se poi volessi arrivare a un milione o un miliardo non sarebbe umanamente possibile.

Ecco che nasce il ciclo `while` per permettere di eseguire un'istruzione piu' volte.

```
i: int = 1
while i <= 1000:
 print(i)
 i = i + 1
```

Con questo piccolo pezzo di codice riesco a stampare i numeri da 1 a 1000; l'idea e' che il `while` ha una condizione booleana (allo stesso modo della selezione `if`) e "fin tanto che" la condizione e' vera (`True`) il blocco di codice successivo viene iterato / ri-eseguito. Nell'esempio la variabile `i`

parte da **1** e ad ogni ciclo viene stampata e incrementata di 1 (quindi il nostro programma stamperà 1, 2, 3, 4, 5, ... 1000).

Per creare un ciclo infinito basta inserire una condizione sempre vera per esempio:

```
while True:
 print("questo e' un ciclo infinito!")
```

## break

E' possibile bloccare un ciclo anche all'interno del suo corpo, non solo tramite la condizione iniziale (detta guardia), tramite la keyword **break** ("rompi il ciclo").

```
i: int = 1
while True:
 if i > 1000:
 break
 print(i)
 i = i + 1
```

In questo caso ottengo lo stesso risultato di prima ma la condizione e' stata spostata dentro il corpo del ciclo. Questo metodo può essere comodo nei casi in cui e' difficile esprimere la condizione a priori o la condizione e' composta da vari fattori indipendenti.

## Esempi

```
stampa i numeri da 1 a 100 usando il ciclo while
x: int = 1
while x <= 100:
 print(x)
 x = x + 1

stampa i numeri da 10 a 100 usando il ciclo while
x: int = 10
while x <= 100:
 print(x)
 x += 1

stampa i numeri pari da 1 a 100
x: int = 1
while x <= 100:
 if x % 2 == 0:
 print(x)
 x += 1
```

# liste

---

Oltre ai tipi di dato nativi (`int`, `float`, `bool`, `str`) esistono anche le liste (`list`).

Le liste non sono altro che un modo per creare una sola variabile che puo' contenere piu' valori. I valori possono essere di qualsiasi tipo. La cardinalita' della lista puo' essere 0 o 1 o 2 o 3, ...

```
lista vuota
l: list = []

lista con un elemento intero
l: list = [1]

lista con 4 elementi interi
l: list = [1,2,4,8]

lista con 3 elementi booleani
l: list = [True, False, True]

lista eterogenea DA EVITARE!
l: list = [3.14, True, 0, "hello", [1,2]]
```

Il tipo di una lista dovrebbe essere `list` seguito dal tipo dei suoi elementi

```
l: list[int] = [1,2,3]
```

Le liste sono indicizzate allo stesso modo delle stringhe (infatti possiamo pensare che le stringhe siano delle liste di caratteri).

```
l: list[int] = [1,2,4,8,16]

stampa il primo elemento
print(l[0])

stampa l'ultimo elemento
print(l[-1])

stampa dal secondo al quarto elemento
print(l[1:4])

stampa gli ultimi tre elementi
print(l[-3:])
```

Le liste sono oggetti (approfondiremo nei prossimi moduli cosa sia un oggetto) e hanno a disposizione delle funzioni (metodi) integrate e accessibili tramite `.`

```
l: list[int] = []

aggiunge un elemento alla lista
l.append(1)
l.append(2)

aggiunge una lista alla lista
l.extend([3,4])
print(l)
```

## for

---

Il `for` e' un secondo modo di ottenere un ciclo/delle iterazioni (meno "potente" del `while`, puo' fare meno cose, ma piu' semplice da scrivere).

Il `for` nasce per ciclare/iterare un iterabile (come una lista) che gli viene passata in input.

```
lista: list[int] = [1,2,3]
for elemento in lista:
 print(elemento)

for elemento in [1,2,3]:
 print(elemento)
```

Spesso si usa la funzione `list(range())` per inizializzare una lista di valori e poi ciclarla con `for`

```
range(100) ritorna i numeri da 0 a 99
lista: list[int] = list(range(100))
for elemento in lista:
 print(elemento)
```

## Esempi

```
stampa i numeri da 1 a 100 usando range e for
for i in range(1,101):
 print(i)

stampa i numeri da 10 a 100 usando range e for
for i in range(10,101):
```

```
print(i)
```

```
stampa i numeri pari da 1 a 100
for i in range(1,101):
 if i % 2 == 0:
 print(i)
```